Two Parallelisms in One Weekend
A Workflow Paper for the Darwin Self-Improving Chess Engine

Point72 Hackathon
Supplement to the Darwin proposal

Abstract

Darwin is a self-improving chess engine: each generation, a strategist proposes improvements to the reigning champion, parallel builders implement them, a validator gates the candidates, a round-robin tournament plays them, and the top two engines advance. It was built in a weekend hackathon by five engineers, each driving a separate AI coding agent against a small set of frozen contracts. This paper documents the *workflow* rather than the engine: the two parallelisms (engine-time and build-time) that the project was structured around, the three contracts that decoupled the workstreams, the per-engineer scopes, the merge sequence that glued it together, and what worked and didn't when the same dual-parallelism shape was applied to humans, agents, and runtime processes simultaneously.

Contents

1 The Two Parallelisms **2**

2 Frozen Contracts as Parallelization Seams **3**
2.1 Engine Protocol . . . 4
2.2 DB Schema . . . 4
2.3 WebSocket Events . . . 4

3 Per-Engineer Workstreams (A–E) **4**
3.1 Person A — Engine Core & Baseline . . . 4
3.2 Person B — Tournament & Referee . . . 4
3.3 Person C — Strategist + Builder Agents . . . 5
3.4 Person D — Frontend Dashboard . . . 5
3.5 Person E — Infra, API, Orchestration, Demo . . . 5

4 The Runtime Agent Pipeline **5**

5 Merge Order and Integration **6**

6 What Worked, What Didn't, What We'd Change **7**
6.1 What worked . . . 7
6.2 What didn't . . . 7

6.3 What we'd change 8

1 The Two Parallelisms

The shape of Darwin is the same shape twice. At runtime, one strategist fans out work to several builders that run in parallel. At build time, five engineers each drove a separate AI coding agent against the same codebase, in parallel, on five branches. We call these *engine-time parallelism* and *build-time parallelism*, and the rest of this paper is organised around the observation that they share the same critical-path ingredient: a small set of frozen interfaces that let independent producers and consumers proceed without waiting on each other.

Engine-time parallelism (Figure 1) is the runtime loop. The reigning champion is shown to a strategist agent; the strategist returns a fixed-shape set of improvement *questions*, each tagged with one of the categories `search`, `evaluation`, `book`, `sampling`, `prompt`. N builder agents run concurrently, one per question, each producing a Python module that satisfies the `Engine` Protocol. A validator runs cheap static gates and a smoke game against `RandomEngine` to drop crashers and no-op fallbacks. The surviving candidates plus the previous-generation incumbents play a round-robin (both colours per pairing, `games_per_pairing` configurable). Selection ranks by tournament score with random tiebreak; the top two engines seed the next generation.

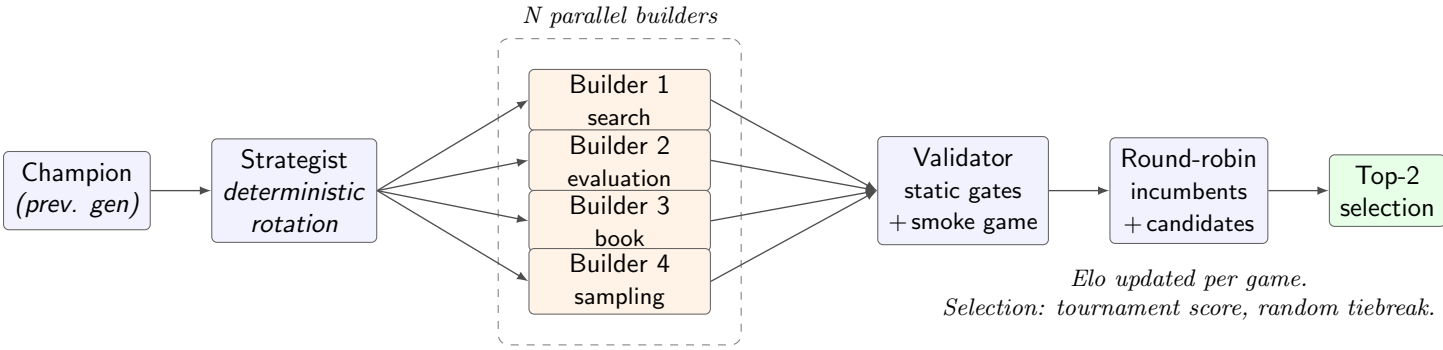

Figure 1: Engine-time parallelism: one strategist fans out to N parallel builders; the validator gates them; the round-robin scores the cohort; the top-2 engines seed the next generation. Source: `backend/darwin/orchestration/generation.py`.

Build-time parallelism (Figure 2) is the human and agent process. Five engineers (Persons A through E), each paired with their own AI coding agent, opened five branches and shipped five workstreams in parallel. The first hour was spent *not* writing code: it was spent agreeing on the three contracts described in §2. Once those were frozen, each engineer wrote against fakes for the other four workstreams, and only integrated when their own piece was real.

The two parallelisms are not analogies. They are the same engineering move in two domains: *commit to a small set of interfaces, then let producers and consumers run concurrently against them.* The runtime loop uses asyncio; the build process uses git branches. The contracts are different files, but the discipline is the same.

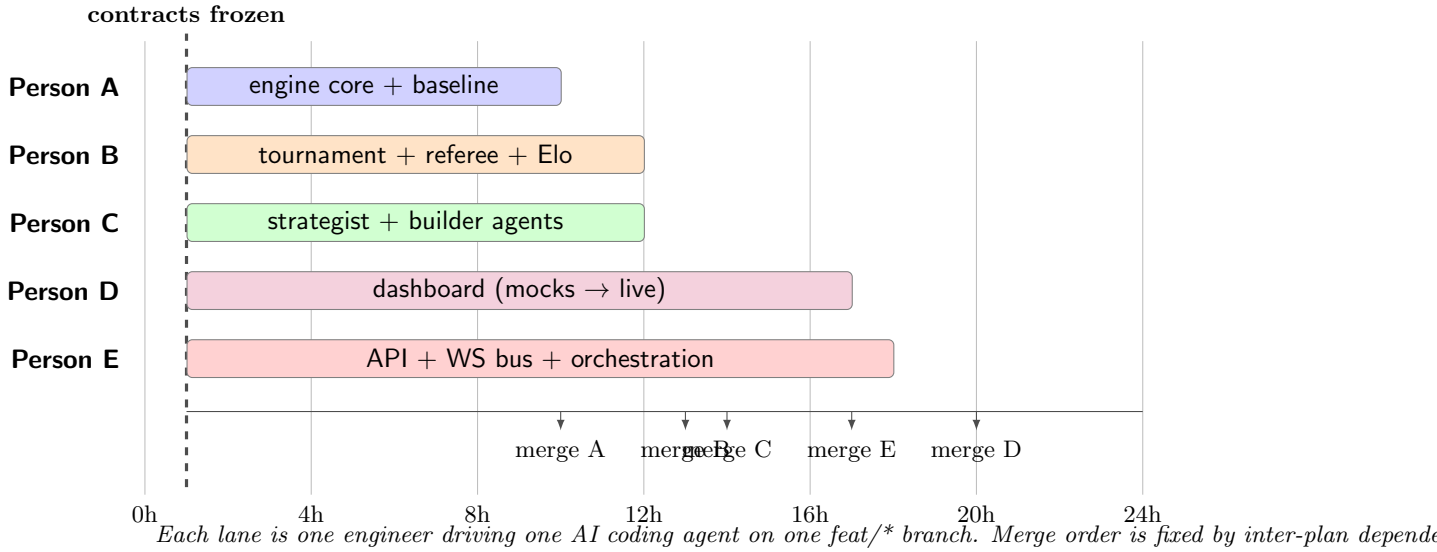

Figure 2: Build-time parallelism: five engineer/agent pairs working against frozen contracts on parallel `feat/*` branches over a 24-hour build window. Merge order is fixed by inter-plan dependencies.

2 Frozen Contracts as Parallelization Seams

Three files were declared frozen at hour 1 and not modified again without team sync. Each one cleanly decoupled a producer from a consumer; together they were enough to let the five workstreams ship in parallel. Figure 3 summarises them.

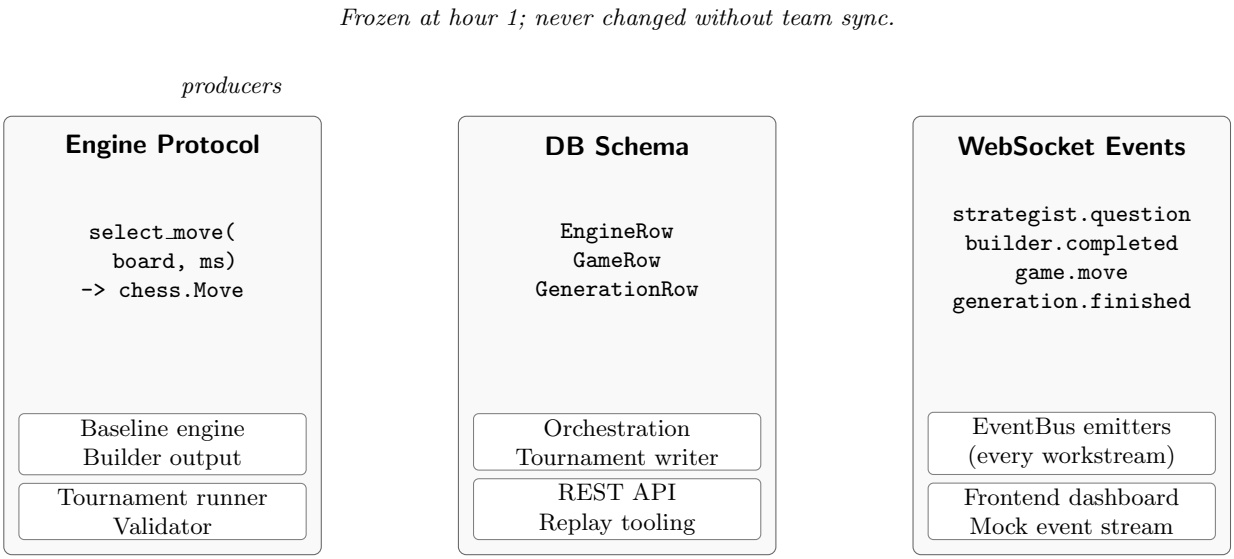

Figure 3: The three frozen contracts and what each decouples. The Engine Protocol decouples engine producers from the tournament runner. The DB schema decouples the orchestrator from the API. The WebSocket events decouple the backend from the frontend.

2.1 Engine Protocol

backend/darwin/engines/base.py. A `runtime_checkable` `Protocol` with three attributes (`name`, `generation`, `lineage`) and one async method, `select_move(board, time_remaining_ms) -> chess.Move`. This is the seam that lets a builder agent emit a Python module at runtime that drops into the same tournament runner as the hand-written baseline. It also lets Person B (tournament) write tests against `RandomEngine` on day one, before Person A's `BaselineEngine` existed, and before any builder-generated engine had been produced. The Protocol's smallness is the point: the fewer fields, the fewer ways the contract can drift.

2.2 DB Schema

backend/darwin/storage/models.py. Three SQLModel tables: `EngineRow`, `GameRow`, `GenerationRow`. The orchestrator writes to all three; the REST API reads from all three; the replay tool reads from the first two and re-emits WebSocket events. Freezing this schema let Person D's frontend mock generation traces against fake JSON, and let Person B's tournament runner write game records before Person E's API existed to read them.

2.3 WebSocket Events

backend/darwin/api/websocket.py on the backend, mirrored in frontend/src/api/events.ts on the frontend. A discriminated union over event types (`generation.started`, `strategist.question`, `builder.completed`, `game.move`, `game.finished`, `generation.finished`, plus cancellation and state-clear events added in follow-ups). Both files are FROZEN comments at the top; the two must stay aligned, and any change requires a paired edit. This is the contract that let Person D ship a fully working dashboard *against a hand-written mock event stream* for most of the hackathon, then flip a query-string flag when Person E's bus came up.

The pattern across all three contracts: *shape committed early, both sides mock against it, real implementations land at their own pace.* The cost of disagreement was paid once, at hour 1, instead of recurring at every integration.

3 Per-Engineer Workstreams (A–E)

The five workstreams below each owned one directory tree and one `feat/*` branch. Plan documents in plans/ define the scopes; this section abstracts them into a half-page each.

3.1 Person A — Engine Core & Baseline

Branch `feat/engine-core`; plan plans/person-a-engine-core.md. Owns the `Engine` Protocol, the `BaseLLMEngine` convenience class, the generation-0 `BaselineEngine`, the `RandomEngine` sparring partner, and the `registry.load_engine` dynamic loader. *Critical property:* every other workstream depends on this Protocol, so shipping a frozen `base.py` with a stub baseline early was the unblocker for the whole project. Tests at this level are tight: a single legal-move assertion for the baseline plus four registry tests covering both dotted-module and filesystem-path loads.

3.2 Person B — Tournament & Referee

Branch `feat/tournament`; plan plans/person-b-tournament.md. Owns `play_game` (one game with per-move time control and graceful loss adjudication), `round_robin` (parallel pairing schedule

via `asyncio.gather`, capped by a semaphore added in follow-up 1), the Elo update (`K=32`), and the `select_top_n` anti-regression gate. The whole pipeline's runtime budget — whether 3 generations can finish in a hackathon — lives or dies on this module's parallelism, which is why Person B was also responsible for the `eval_match.py` headline-number CLI.

3.3 Person C — Strategist + Builder Agents

Branch `feat/agents`; plan `plans/person-c-agents.md`. Owns `propose_questions` (the strategist) and `build_engine`/`validate_engine` (the builder). The strategist returns 2–4 distinct improvement questions, one per category; the builder calls a `submit_engine` tool whose `input_schema` forces a single-string `code` field. The validator runs static gates (forbidden-import regex, required-pattern regex, `ast.parse`) and a smoke game against `RandomEngine`. Person C's stub implementations were the single most important early ship: they let Person E wire the orchestrator end-to-end against fakes hours before the real agents were ready.

3.4 Person D — Frontend Dashboard

Branch `feat/frontend`; plan `plans/person-d-frontend.md`. Owns the React + Vite + Tailwind dashboard, including `LiveBoard`, `EloChart`, `StrategistFeed`, `Bracket`, and `GenerationTimeline`. Built against `src/hooks/mockEvents.ts` for offline development and switched to the live WebSocket via a `?mock=1` query-string flag once Person E shipped the bus. The flag was the entire integration: the event shapes were guaranteed by the contract.

3.5 Person E — Infra, API, Orchestration, Demo

Branch `feat/infra`; plan `plans/person-e-infra.md`. Owns the integrator's pile: the SQLite database, the FastAPI server, the WebSocket bus, the top-level `orchestration/generation.py` loop that calls into A, B, and C, the shared LLM helper with semaphore + retry, the Procfile, the seed and replay scripts, and the demo plan. Person E is the only role that touches every other workstream; the corresponding mitigation is that they touch *only* through the three frozen contracts.

4 The Runtime Agent Pipeline

The orchestration loop in `generation.py` is the engine-time parallelism made concrete. One generation, in order:

1. **Pool warm-up.** `warm_modal_pool(40)` kicks off 40 Modal containers in the background while the strategist and builders run. This is a small but important asynchrony: the ∼25–30 seconds spent on agent calls is enough for the warm pool to be ready by the time the tournament dispatches.

2. **Strategist.** `propose_questions(champion_src, history)` returns a list of `Question` objects, each with a category and a free-text hypothesis. Reads the champion's source via `inspect.getsource` so the agent prompts are grounded in the actual code under modification, not an abstract description.

3. **Parallel builders.** `asyncio.gather(*[build_engine(...) for q in questions])`. Each builder calls the builder LLM with the `submit_engine` tool, runs the static gates, and writes `engines/generated/<name>.py`.

4. **Validator.** Each candidate is loaded through the registry and plays one short game against `RandomEngine`. Crashers, illegal-move emitters, and time-outers are dropped before they reach the tournament. The `builder.completed` WebSocket event fires the moment each candidate's verdict is known, so the dashboard sees results stream in rather than batched at the end.

5. **Round-robin.** `round_robin(cohort, games_per_pairing, time_per_move_ms, on_event=bus.emit)`. The cohort is incumbents ∪ accepted candidates — typically 4–6 engines, so 60–90 games. The `max_parallel_games` semaphore caps actual concurrency to keep the LLM provider's quota in line.

6. **Elo update.** `update_ratings_for_games(ratings, standings.games)` applies `K=32` per game. New candidates seed at 1500.

7. **Selection.** `select_top_n` ranks by tournament score with random tiebreak; the primary incumbent is the anti-regression baseline.

8. **Persist + emit.** A `GenerationRow` with the strategist questions, a `GameRow` per game, an `EngineRow` per accepted candidate, and a final `generation.finished` event with the post-tournament Elo for every cohort member.

The whole-system architecture, including how this loop hangs off the API, the database, and the optional Modal backend, is shown in Figure 4.

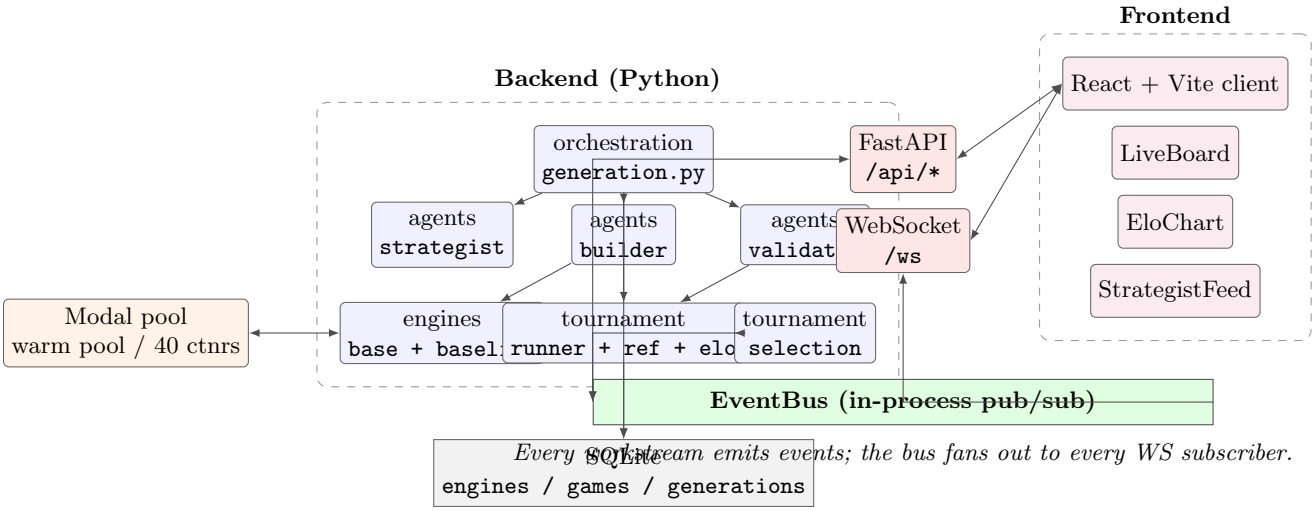

Figure 4: Whole-system architecture: backend modules, SQLite store, event bus, optional Modal pool for engine execution, FastAPI surface, React frontend.

5 Merge Order and Integration

Inter-plan dependencies dictated a strict merge order. From `plans/README.md`:

1. **Person A merges `feat/engine-core`.** The Protocol and `RandomEngine` go live; B and C unblock.

2. **Person B merges `feat/tournament`.** The runner exists in main; E can wire orchestration against real games.

3. **Person C merges `feat/agents`.** Real strategist and builder land; E's orchestration runs end-to-end against real agents.
4. **Person E merges `feat/infra`.** API + WS bus go live; D switches off mocks.
5. **Person D merges `feat/frontend`.** Demo-ready.

The order is not a *decoration*; each merge unblocks the next one. But because each downstream consumer was already mocking against the frozen contract, no upstream merge *forced* a downstream rewrite. A second merge of `feat/agents` that subtly broke the `Question` dataclass would still ripple, but the contract was designed to make that ripple narrow: the union of fields the orchestrator reads off a `Question` is `index`, `category`, `text`.

Four follow-up branches landed after the initial five: tournament concurrency, builder quality gate, champion resume, and frontend polish. Each was scoped to a single workstream's directory tree — by construction, since the frozen contracts were already covering the seams.

6 What Worked, What Didn't, What We'd Change

6.1 What worked

Hour-1 contract freeze. Pre-committing the three contracts made the rest of the day a sequence of independent local optimisations. None of the workstreams were blocked on inter-team coordination between hour 2 and hour 16.

Mocks before reals. Person D's mock event stream and Person C's stub agents both made their dependents productive *before* the real implementations existed. The pattern was explicit in every plan: ship a stub, then ship the real.

The validator's smoke game. Catching a builder-generated engine that times out, returns illegal moves, or crashes on the opening position — before it can pollute the tournament — saved generations that would otherwise have been pure noise.

Per-event streaming on the bus. The dashboard sees `builder.completed` fire as each candidate's verdict resolves, not in a batch. The loop felt live during demos, not deferred.

6.2 What didn't

Builder hallucinated import. Every builder-generated engine in early generations contained `from darwin import config as settings`, which imports the *module* not the settings instance and then silently raises `AttributeError` caught by the engine's own fallback. Net effect: the engine plays `next(iter(legal_moves))` forever, the validator's smoke game passes, the tournament is meaningless. Follow-up 2 added the explicit-imports section to the prompt and a banned-import regex on the validator side. Lesson: *a fallback that hides bugs is worse than a crash.*

Tournament concurrency self-immolation. A `round_robin` that fires every pairing in parallel will, on a slow LLM provider, stack moves until the per-move 25 s timeout trips on every game and every game terminates on `time`. The "winner" becomes whichever bug fires last. Follow-up 1 added a `max_parallel_games` semaphore (default 2). Lesson: *parallelism on top of a rate-limited resource needs to be gated, not multiplied.*

API path didn't resume the lineage. `run_generation_task` hard-coded the baseline as the champion for every API-triggered run. The CLI threaded the champion forward correctly, but `POST /api/generations/run` would restart from baseline-v0 every time, breaking the entire self-improvement premise. Follow-up 3 loaded the previous `GenerationRow.champion_after` from the DB. Lesson: *having two entry points to the same loop is two contracts to keep aligned.*

Frontend hid termination reasons. The `generation.finished` event carried `promoted=true` or `false`; the UI rendered "KEPT" or "PROMOTED" badges. A generation in which every game terminated on `time` looked visually *successful* on the dashboard. Follow-up 4 added a termination breakdown to the timeline. Lesson: *a successful UI state should require a successful underlying run, not just a non-error one.*

6.3 What we'd change

Make the strategist deterministic earlier. The pure-code branch's strategist is a deterministic rotation through canned question pools, no LLM call. This is more reproducible than an LLM-generated strategist and faster — but the original plan had it as an LLM call, so there was a wasted half-day of API budget on strategist queries that produced no signal a canned rotation wouldn't have produced. We'd start deterministic and add LLM generation only if the canned pool ran out.

One contract, one file. The WebSocket events are split across `websocket.py` and `events.ts`. They drifted twice and the second drift was caught only by the frontend logging "unknown event type". A code-generation step would prevent this; absent that, a CI check that compares the two file's union members would.

Per-workstream test partition. Tests live in one `backend/tests/` directory, but each one belongs to one workstream. Treating tests as part of the workstream's directory tree (e.g. `backend/darwin/tournamen` would make the ownership clearer and let the CI matrix run only the affected workstream's tests on a touched branch.

Conclusion

The dual-parallelism framing isn't aesthetic. It was the operating discipline. Engine-time parallelism is asyncio against a Protocol; build-time parallelism is git branches against a contract. In both cases the work that mattered most was the work that wasn't writing the parallel code: it was committing, ahead of time, to the small set of interfaces that let the parallel code exist.